

1-搜索算法

搜索算法

无信息搜索

- DFS/BFS
- UCS
- IDDFS

有信息搜索

- Greedy
- A-star

这节课是CS188的第一部分，以PacMan为例子讲解搜索算法，属于Planning规划问题。下一节课讲解CSP问题[2-CSP BackTracking](#)和[3-CSP LocalSearch](#)，算法实现有很大不同，但也归类到搜索问题

规划问题的特点是

- 状态的具体细节不可知，只能判断是不是目标状态 bool
- 关心如何到达目标状态，也就是路径
- 通过一个后继函数得到下一个状态

一些概念

一开始只能将初始节点放入边缘，然后循环，只要边缘非空（说明还有路径可以走）或者没达到目标：

从边缘中取出节点，放入已扩展的节点中 - 生成这个节点的后继节点并放入边缘 - 取出节点...

边缘 所有下一步可以选择的节点的总和，可以用优先队列统一存储

扩展 将边缘中的一个节点取出（选择走到这一个位置）

生成 计算取出节点的后继，并放入边缘

伪代码

```
def GRAPH-SEARCH(problem, frontier): #returns a solution, or failure
    closed #空集合 存储已访问的节点防止陷入循环
    frontier #边缘
    action #存储进行的操作

    frontier.insert(MAKE-NODE(INITIAL-STATE[problem]), frontier) #将初始节点
放入边缘
    closed.append(INITIAL-STATE[problem]);
```

```

loop do:
  if frontier is empty:
    return failure

  node <- REMOVE-FRONT(frontier) #从边缘中取出一个节点 关键!
  if isGoal(problem, STATE[node]): #取出的节点如果是目标, 返回
    return action
  if STATE[node] is not in closed: #是未访问的
    add STATE[node] to closed
    frontier.insert(EXPAND(node, problem), frontier)
    action.add(node)
end

```

不同的搜索算法都遵循这个框架，区别是

1. 如何决定从边缘中取出哪个节点/边缘的数据结构
2. 如何评判 if STATE[node] is not in closed? 如果有个更低代价的路线，但是走过，是不是要再走？

无信息搜索

除了判断扩展出的节点是不是目标（布尔值）外没有关于目标的额外信息

BFS 广度优先搜索

```

def breadthFirstSearch(problem):
    # 采用utils提供的数据结构，构建一个队列
    Frontier = util.Queue()
    # 创建已访问节点集合
    Visited = []
    # 将(初始节点, 空动作序列)入队
    Frontier.push( (problem.getStartState(), []) )
    # 将初始节点标记为已访问节点
    Visited.append( problem.getStartState() )
    # 判断队列非空
    while Frontier.isEmpty() == 0:
        # 从队列中弹出一个状态和动作序列
        state, actions = Frontier.pop()
        # 判断是否为目标状态，若是则返回到达该状态的累计动作序列
        if problem.isGoalState(state):
            return actions
        # 遍历所有后继状态
        for next in problem.getSuccessors(state):
            # 新的后继状态
            n_state = next[0]
            # 新的action
            n_direction = next[1]

```

```
# 若该状态没有访问过
```

```
if n_state not in Visited:  
    # 计算到该状态的动作序列，入队  
    Frontier.push( (n_state, actions + [n_direction]) )  
    Visited.append( n_state )
```

边缘的数据结构：栈 后进先出LIFO

每次新生的后继节点都在栈顶，这样永远先扩展新的节点，也就会一路走下去

完备，但是不一定找到最优解

内存占用小

DFS 深度优先搜索

```
def depthFirstSearch(problem):  
    Frontier = util.Stack()  
    Visited = []  
    Frontier.push( (problem.getStartState(), []) )  
    Visited.append( problem.getStartState() )  
    while Frontier.isEmpty() == 0:  
        state, actions = Frontier.pop()  
        if problem.isGoalState(state):  
            return actions  
        for next in problem.getSuccessors(state):  
            n_state = next[0]  
            n_direction = next[1]  
            if n_state not in Visited:  
                Frontier.push( (n_state, actions + [n_direction]) )  
                Visited.append( n_state )
```

和DFS的唯一区别，边缘的数据结构：队列 先进先出FIFO

扩展的节点会在之前已有的节点访问后再访问，保证先搜索完一层才进入下一层

完备，最优步数

内存占用大

UCS 代价一致搜索

如果主要考虑走一步的代价

```
def uniformCostSearch(problem):  
    """Search the node of least total cost first."""  
    #always pop the min priority member, aka min cost member  
    Frontier = util.PriorityQueue()  
    VisitedMinCost = {}  
    Frontier.push( (problem.getStartState(), [], 0), 0 ) #push除了状态、
```

action, 还有代价

```
VisitedMinCost[problem.getStartState()] = 0

while Frontier.isEmpty() == 0:
    state, actions, cost = Frontier.pop()
    if state in VisitedMinCost and cost > VisitedMinCost[state]: #区别2
        continue
    if problem.isGoalState(state):
        return actions
    for next in problem.getSuccessors(state):
        n_state = next[0]
        n_direction = next[1]
        add_cost = next[2]

        n_cost = cost + add_cost #更新cost
        if n_state not in VisitedMinCost or n_cost <
VisitedMinCost[n_state]: #区别2
            Frontier.push( (n_state, actions + [n_direction], n_cost),
n_cost )
            VisitedMinCost[n_state] = n_cost
```

区别1 边缘的数据结构：优先队列 会按照优先级进行排序
先选择边缘中**累计成本**最低的节点扩展

区别2 访问过的节点，如果代价更低，也要继续访问
所以Visited不是之前的集合而是字典/哈希表（state -> cost）

完备、最优（按着最低成本的路线一个个找，总能找到最低成本的目标路线）
内存占用大

IDDFS 迭代加深的深度优先搜索

```
def iterativeDeepeningSearch(problem):
    """
    Iterative deepening search combines the space efficiency of depth-first
    search
    with the optimality of breadth-first search. It performs a series of
    depth-limited
    searches with increasing depth limits until a solution is found.
    """
    def depthLimitedSearch(problem, limit):
        """
        Helper function to perform depth-limited search with a given depth
        limit.
        Returns the solution path if found within the limit, None
        otherwise.
        """
```

```

Frontier = util.Stack()
Frontier.push( (problem.getStartState(), [], 0,
{problem.getStartState()}) )

while Frontier.isEmpty() == 0:
    state, actions, cur_depth, visited = Frontier.pop()
    for next in problem.getSuccessors(state):
        n_depth = cur_depth + 1
        n_state = next[0]
        n_direction = next[1]

        if problem.isGoalState(n_state):
            return actions + [n_direction]

        if n_depth < limit and n_state not in visited: #限制最大深度
            n_visited = set(visited) #每个路线都有自己的visited集合 而
            #不是共用的 防止不同路线之间冲突
            n_visited.add(n_state)
            Frontier.push( (n_state, actions + [n_direction],
n_depth, n_visited) )
        return None

depth_limit = 0
while True:
    result = depthLimitedSearch(problem, depth_limit)
    if result is not None:
        return result
    depth_limit += 1 #每次都增加一个深度
    if depth_limit > MAX_DEPTH: # maximum
        break

```

边缘的数据结构：栈

深度逐渐增加的DFS，实际上结合了BFS和DFS的特点

完备，最优

会重复搜索开头部分的节点，不过开头部分的搜索树也比较小，搜索的总节点数量可能是DFS/BFS的几百倍

占用内存小（接近DFS）

有信息搜索

greedy search 贪心

和UCS一样，只不过从选最小代价的节点变为选最小启发式函数值的节点

启发式函数是人为设定的，评估**和结果的接近程度**，要求函数值是非负的（目标的值为0）

A-star

结合了前溯累计代价和未来启发式函数值，综合得到的最好的无信息搜索算法

根据 $f(x) = g(x) + h(x)$ 最小的来选择下一个扩展的节点

g 为累计的成本， h 为启发式函数的值

```
def aStarSearch(problem, heuristic=manhattanHeuristic):
    """Search the node that has the lowest combined cost and heuristic
    first."""
    Frontier = util.PriorityQueue()
    VisitedMinGCost = {}
    startState = problem.getStartState()
    #frontier member [2] = combined cost(g), priority = f = g + h
    Frontier.push( (startState, [], 0), 0 + heuristic(startState,problem) )
    VisitedMinGCost[startState] = 0

    while Frontier.isEmpty() == 0:
        state, actions, g_cost = Frontier.pop()
        if state in VisitedMinGCost and g_cost > VisitedMinGCost[state]:
            continue
        if problem.isGoalState(state):
            return actions
        for next in problem.getSuccessors(state):
            n_state = next[0]
            n_direction = next[1]
            add_g_cost = next[2]

            n_g_cost = g_cost + add_g_cost
            if n_state not in VisitedMinGCost or n_g_cost <
VisitedMinGCost[n_state]:
                n_f_cost = n_g_cost + heuristic(n_state,problem)
                Frontier.push( (n_state, actions + [n_direction],
n_g_cost), n_f_cost )
                VisitedMinGCost[n_state] = n_g_cost
```

注意启发式函数的设计，除了满足非负，还要满足

1. 可采纳性 Admissibility

$h(x) \leq \text{actual cost from } x \text{ to GOAL}$

否则不一定找得到最优路径

2. 一致性 Consistency

$h(A) - h(C) \leq \text{cost from } A \text{ to } C$

如果不满足一致性 (Consistency)，但满足可采纳性 (Admissibility)。对于“树搜索”(Tree Search，不记录已访问节点)：A 仍然能保证找到最优解；对于“图搜索”(Graph Search，记录已访问节点/使用 Closed Set)：A 可能会失去最优性，找到的路径可能不是最短的。

actual cost from x to GOAL

和之前UCS的成本是不一样的，之前的累计成本是从起点开始计算的，而这里是离终点还有多少成本，其实就是理想的启发式函数的值

这种理论的启发式函数是存在的，只是我们没法知道（不然就无需搜索，直接知道最优路径了）

一般来说，越接近理想的启发式函数，可能的启发式函数计算量也会越大，但是搜索花费的步数会更小（理想的情况是直接走出最优路径）

一个平凡的启发式函数就是 $h(x)=0$ （的确满足两个性质），此时退化为UCS（步数最大）